Table des matières

Exercices corrigés 02 — Ranger les fichiers en bronze 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 3
Mises en situation 3
Corrigé 5

Exercices corrigés 02 — Ranger les fichiers en bronze

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 02 et du Labo 02.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Ligne ou colonne. Pour chaque usage, dites quel stockage convient : (a) l'ERP qui enregistre une commande ; (b) la vibration moyenne par étiquette sur deux ans ; (c) le portail qui met à jour le statut d'une réclamation ; (d) le total des ventes par gamme et par mois.
2. Deux colonnes sur quatre. Une requête ne lit que `TagName` et `Value`. Donnez deux raisons pour lesquelles elle va plus vite sur un format en colonnes.
3. Échange ou stockage. Pourquoi le CSV reste-t-il le format d'export de tant de systèmes, alors qu'il fait un mauvais format de lac ?
4. Avro ou Parquet. Lequel convient à un flux d'événements écrits un à un, lequel à un lac analytique, et pourquoi ?
5. Les trois niveaux. Nommez les trois niveaux d'un fichier Parquet, et dites à quoi sert chacun.
6. Le pied de page. Que contient le pied de page d'un fichier Parquet, et pourquoi un moteur le lit-il avant le reste ?
7. Encodage et compression. Quelle est la différence ? Donnez un exemple de chacun sur l'historien.
8. Snappy ou zstd. Sur l'historien en un seul fichier, snappy donne 1,8 Mo et zstd 1,2 Mo. Qu'est-ce qui pourrait faire choisir snappy malgré tout ?

© 2026 iAlexMG. Tous droits réservés.

9. Projection et prédicat poussé. Définissez l'un et l'autre en une phrase.
10. Des statistiques inutiles. Pourquoi le minimum et le maximum de `TagName` ne font-ils sauter aucun groupe de lignes dans un fichier quotidien de l'historien ? Que faudrait-il changer ?
11. La règle du bronze. Énoncez-la en une phrase. Quel type reçoivent `Timestamp` dans l'historien et `recu_le` dans la télémétrie, et pourquoi pas le même ?
12. Le montant. Pourquoi un montant ne se range-t-il jamais en `DOUBLE` ?
13. Le stockage objet. Nommez deux différences entre un stockage objet et un disque qui changent la façon de ranger un lac.
14. L'élagage. Qu'est-ce que l'élagage des partitions, et en quoi va-t-il plus loin que le prédicat poussé ?
15. La clé de partition. Donnez les trois critères du choix d'une clé, et appliquez-les à l'historien de Boréal.
16. Les petits fichiers. Pourquoi 843 fichiers de 13 Ko se lisent-ils plus lentement qu'un seul fichier de même contenu ? Comment s'appelle le remède ?
17. [labo] Le jour livré. Dans quelle partition du bronze vivent les valeurs de départ datées du 30 septembre 2025 ? Pourquoi ce choix prépare-t-il le Cours 03 ?
18. [labo] Les dossiers. Que répond DuckDB si `plateforme/lac/bronze/usine/` n'existe pas au moment du `COPY`, et que faut-il en retenir ?
19. [labo] Relancer. Que se passe-t-il si vous relancez `bronze_historien.sql` ? Et si vous retirez `OVERWRITE` avant de le relancer ?
20. [labo] Les mesures. Donnez la taille de l'historien en CSV et dans le bronze, et le nombre de fichiers que lit une requête filtrée sur `jour = DATE '2025-10-28'`.

1. [labo] Une seule colonne `json`. Pourquoi `read_json('sources/telemetrie/*.jsonl')` ne donne-t-il qu'une colonne `json`, et quelles sont les deux façons de l'éviter ?
2. [labo] Deux températures. Pourquoi le bronze de la télémétrie garde-t-il `temp_moteur_max` et `temperature_moteur_max` en deux colonnes, au lieu de les réunir ?

Questions pratiques

Ici on écrit vraiment : des commandes, du SQL. Tapez votre réponse avant de la comparer. Une requête qu'on croit connaître et qu'on n'a jamais écrite de mémoire n'est pas acquise.

P1. Le pied de page d'un jour. Écrivez la requête qui affiche, pour la partition du 2025-10-28, le nombre de lignes du groupe et le minimum et le maximum de `Timestamp`, sans lire les données. Expliquez le minimum.

P2. Le poids de chaque colonne. Pour la même partition, écrivez la requête qui donne la taille compressée et non compressée de chaque colonne. Quelle colonne pèse le plus, laquelle le moins, et pourquoi ?

© 2026 iAlexMG. Tous droits réservés.

P3. Filtrer sur la partition ou sur l'horodatage. Écrivez deux requêtes `EXPLAIN ANALYZE` qui comptent les lignes du 1er octobre 2025 dans le bronze de l'historien : l'une filtre `jour`, l'autre filtre `Timestamp`. Combien de fichiers chacune lit-elle ? Les deux rendent 3 714 lignes : sont-ce les mêmes ? Prouvez-le par une requête.

P4. Trier pour que les statistiques servent. Créez un dossier `essais` dans `Projet_Boreal`, puis écrivez le `COPY` qui range tout le bronze de l'historien dans un seul fichier zstd, `essais/historien_trie.parquet`, trié par étiquette puis par horodatage. Lisez les statistiques de `TagName` dans ses groupes de lignes : combien de groupes une requête sur `UTL.CMP2.VIBRATION` doit-elle lire ? Comparez la taille du fichier à celle d'un fichier non trié (1,2 Mo, Théorie 02) et expliquez l'écart.

P5. Compacter la télémétrie. Écrivez le `COPY` qui compacte le bronze de la télémétrie dans un seul fichier zstd, `essais/telemetrie_compacte.parquet`. Comparez sa taille à celle des 843 fichiers, puis le temps d'un `COUNT(DISTINCT message.sernr)` sur l'un et sur l'autre.

Diagnostics

On vous montre quelque chose qui ne va pas. Dites ce qui cloche, pourquoi, et donnez le correctif. Ces situations sont toutes arrivées à quelqu'un.

E1. Un script d'équipe regénère les sources avec `--historien complet` pour préparer une étude annuelle, puis lance l'étude sur le bronze de l'historien. Aucune erreur. L'étude ne montre pourtant que trois mois.

E2. Pour « tout rendre filtrable », une collègue partitionne le bronze de l'historien par étiquette et par jour : `etiquette=…/jour=…/`. Le lac compte maintenant des milliers de fichiers minuscules, et une requête sur un mois entier est devenue plus lente qu'avant.

Questions à développer

Ces questions ne se répondent pas en un mot : on veut un court paragraphe de raisonnement. Rédigez votre réponse avant d'ouvrir le corrigé.

A. D'où vient la vitesse. Sur l'historien, la même requête prend environ 3 secondes sur les CSV aux types devinés, 0,6 seconde aux types déclarés, et 0,016 seconde sur le bronze. Expliquez ce que chaque écart doit à la détection des types, à la lecture en texte et au format en colonnes.

B. Défendre la règle du bronze. Un collègue veut « tout typer dès le bronze pour faciliter la vie de l'ingénieur analytique » : dates SAP converties, horodatages ramenés à UTC, codes en entiers. Répondez-lui, avec au moins deux exemples tirés de Boréal.

C. Une partition pour les extraits de l'ERP. Les extraits mensuels de l'ERP seront chargés au Cours 03. Proposez une façon de les ranger s'ils passaient par le bronze, en appliquant les trois critères du choix d'une clé, et dites ce que devient l'extrait `2023-03_bis`.

Mises en situation

Le format le plus proche du travail réel. On vous décrit un organisme, ses données et ses contraintes ; à vous de décider et de justifier. Il n'y a pas une seule bonne réponse, mais il y a des réponses indéfendables. Le corrigé donne une solution défendable et dit ce qui serait acceptable aussi.

© 2026 iAlexMG. Tous droits réservés.

M1. Transport collectif des Deux-Rives, le bronze. Les 240 autobus de la société transmettent leur position toutes les 10 secondes, en JSON : jusqu'à deux millions de messages par jour, qu'un service écrit toutes les cinq minutes dans un nouveau fichier. Les analystes interrogent surtout par date et par ligne d'autobus ; la planification relit une année entière chaque automne.

Choisissez le format, la compression, la clé de partition et la taille des fichiers du bronze, et dites ce qu'il faut faire des fichiers écrits toutes les cinq minutes.

M2. Coopérative électrique de la Rivière-aux-Saules, le bronze. Chaque nuit, le système de collecte livre un CSV des lectures de la veille : 18 000 compteurs, une lecture aux 15 minutes, l'heure de fin d'intervalle avec son décalage à UTC, l'énergie en kWh avec trois décimales, et un code de qualité (`R` réel, `E` estimé).

Décrivez le bronze de ces lectures : le type de chaque colonne, la clé de partition, et ce qui pourrait mal tourner si l'on laissait l'outil deviner.

M3. (avancé) Le lac de Boréal dans le nuage. La direction de Boréal veut déplacer le lac dans un stockage objet, facturé à la requête et à l'octet lu. Le bronze compte aujourd'hui 92 fichiers pour l'historien et 843 pour la télémétrie, et la télémétrie grossit d'un fichier par jour. Les analystes lisent surtout les trente derniers jours ; la science des données relit tout chaque semaine.

Proposez une organisation du bronze qui tienne compte du coût, et un plan de compactage. Dites ce qui vous ferait changer d'avis.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés. Les chiffres sont ceux de la génération par défaut de Boréal ; les temps, ceux du poste de l'auteur.

Questions courtes

1.

   (a) En lignes : l'ERP écrit et relit une commande entière à la fois. (b) En colonnes : deux colonnes (`TagName`, `Value`) sur des millions de lignes. (c) En lignes : une mise à jour ponctuelle d'un enregistrement, le terrain des bases transactionnelles. (d) En colonnes : une agrégation qui ne lit que la gamme, le mois et le montant.

Théorie : `Theorie_02_DE.md`, section 1.

2. D'abord, le moteur ne lit que les deux colonnes nommées et laisse les deux autres sur le disque. Ensuite, ces colonnes se compressent bien, parce que des valeurs semblables se suivent : `TagName` ne prend que 58 valeurs. Moins d'octets à lire, et des octets plus petits.

Théorie : `Theorie_02_DE.md`, section 1.

3. Parce qu'il est universel : tout système sait en écrire, tout humain sait en lire, sans bibliothèque ni documentation. Ces qualités en font un bon format d'échange. Pour stocker, il n'a que des défauts : aucun type, aucune compression propre, et une lecture forcée de tout le fichier.

Théorie : `Theorie_02_DE.md`, section 2.

4. Avro pour le flux : c'est un format en lignes, fait pour écrire vite un enregistrement à la fois, avec un schéma embarqué qui prévoit son évolution. Parquet pour le lac : un format en colonnes, fait pour lire vite peu de colonnes sur beaucoup de lignes.

Théorie : `Theorie_02_DE.md`, section 2.

5. Le groupe de lignes, qu'un moteur lit ou saute en entier, et traite en parallèle ; le bloc de colonne, les valeurs d'une colonne dans un groupe, qui permet la lecture colonne par colonne ; la page, une tranche d'un bloc, l'unité d'encodage et de compression.

Théorie : `Theorie_02_DE.md`, section 3.

6. Le schéma (le nom et le type de chaque colonne) et, pour chaque bloc de colonne, le nombre de valeurs, le minimum et le maximum. Le moteur le lit d'abord parce qu'il y apprend les types sans rien deviner, et parce que les statistiques lui disent quels groupes de lignes il peut sauter avant d'en lire un seul octet.

Théorie : `Theorie_02_DE.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

7. L’encodage tire parti de la forme des valeurs, la compression réduit les octets qui en résultent. Encodage : un dictionnaire remplace les 58 étiquettes répétées par de petits numéros ; un codage par plages écrit une seule fois le code de qualité 192 répété des milliers de fois. Compression : zstd ou snappy, appliqués ensuite à chaque page.

Théorie : `Theorie_02_DE.md`, section 3.

8. La vitesse d’écriture et de lecture : snappy compresse et décompresse plus vite. Pour des fichiers écrits sans cesse et lus une fois, ou pour un outil qui ne connaît que snappy, il reste un bon choix. Le bronze de ce parcours se lit bien plus souvent qu’il ne s’écrit : zstd y rapporte davantage.

Théorie : `Theorie_02_DE.md`, section 3.

9. La projection : ne lire que les colonnes que la requête nomme. Le prédicat poussé : comparer le filtre aux statistiques d’un groupe de lignes, et sauter le groupe si aucune ligne ne peut y répondre.

Théorie : `Theorie_02_DE.md`, section 3.

10. Parce que chaque fichier quotidien est écrit dans l’ordre des scrutations : toutes les étiquettes se mélangent dans le même groupe, qui va de `ESS.BANC.REGIME` à `UTL.SEC.POINT_ROSEE`. Aucune étiquette ne tombe hors de cet intervalle. Il faudrait trier les lignes par étiquette à l’écriture : chaque groupe couvrirait alors une plage d’étiquettes étroite (exercice P4).

Théorie : `Theorie_02_DE.md`, section 3.

11. On déclare les types, on ne les devine pas, et l’on ne déclare un type que si la conversion ne perd rien. `Timestamp` devient un `TIMESTAMP` : il a un format fixe et aucun fuseau, et rien ne se perd. `recu_le` reste du texte : le convertir jetterait son décalage.

Théorie : `Theorie_02_DE.md`, section 4.

12. Parce qu’un `DOUBLE` stocke une approximation binaire : la plupart des montants au cent près n’y ont pas de représentation exacte, et une somme peut s’écarter d’un cent. Un montant se range en `DECIMAL`, qui compte exactement les centièmes.

Théorie : `Theorie_02_DE.md`, section 4.

13. Deux parmi trois. Il n’y a pas de vrais dossiers, seulement des clés : lister un dossier est une recherche par préfixe, d’autant plus chère qu’il y a d’objets. Un objet ne se modifie pas, il se remplace en entier : on n’ajoute pas une ligne à un fichier du lac. Et l’on paie à la requête et à l’octet lu : lire moins de fichiers économise de l’argent.

Théorie : `Theorie_02_DE.md`, section 5.

14. L’élagage des partitions écarte des fichiers entiers d’après leur chemin (`jour=2025-10-28/`), sans même les ouvrir. Le prédicat poussé, lui, doit ouvrir chaque fichier pour lire les statistiques de son pied de page avant de sauter un groupe de lignes.

Théorie : `Theorie_02_DE.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.

15. Le grain de livraison, le filtre fréquent, la cardinalité. Pour l'historien : on livre un fichier par jour, qu'on voudra remplacer seul s'il est livré de nouveau ; les consommateurs filtrent par date ; et 92 jours donnent 92 partitions, un nombre raisonnable. La clé est donc le jour livré.

Théorie : `Theorie_02_DE.md`, section 5.

16. Parce que chaque fichier a un coût fixe, quel que soit son contenu : l'ouvrir, lire son pied de page, et, dans un stockage objet, payer une requête. Sur 843 fichiers, ce coût fixe dépasse le coût de la lecture elle-même. Le remède s'appelle le compactage : réécrire les petits fichiers en quelques gros.

Théorie : `Theorie_02_DE.md`, section 5.

17. Dans `jour=2025-10-01` : le fichier de l'historien du 1er octobre commence par ces valeurs de départ, et la partition suit le fichier livré, pas l'horodatage. Au Cours 03, quand un jour est livré de nouveau, on remplace sa partition entière, et elle contient exactement ce que le fichier contenait, rien de plus, rien de moins.

Théorie : `Theorie_02_DE.md`, section 5.

18. `IO Error: Failed to create directory "plateforme/lac/bronze/usine/historien"`. DuckDB crée le dossier final d'un `COPY` partitionné, mais pas ses parents. Un script de bronze suppose donc que l'arborescence du lac existe. Le labo provoque l'erreur exprès en 1.2, pour qu'on la reconnaisse, puis crée les dossiers par `mkdir` en 1.3.

Théorie : `Theorie_02_DE.md`, section 6.

19. Relancé tel quel, le script redonne exactement le même bronze et le même contrôle (275 436 des deux côtés) : `OVERWRITE` vide le dossier avant d'écrire. Sans `OVERWRITE`, DuckDB refuse d'écrire dans un dossier qui n'est pas vide : `Directory "plateforme/lac/bronze/usine/historien" is not empty!`.

Théorie : `Theorie_02_DE.md`, section 6.

20. 13 725 380 octets en CSV, 1 468 497 octets dans le bronze, neuf fois moins. La requête filtrée sur le 28 octobre lit un fichier : `Scanning Files: 1/92`.

Théorie : `Theorie_02_DE.md`, sections 2 et 5.

21. Les 39 premiers jours de l'archive, du 2023-08-01 au 2023-09-08, sont des fichiers vides. La détection échantillonne les premiers fichiers, n'y trouve rien, et se rabat sur une colonne `json` brute. Deux façons de l'éviter : déclarer les colonnes (`columns = {…}`), comme le script du bronze ; ou ajouter `union_by_name = true`, qui fait lire la structure de chaque fichier.

Théorie : `Theorie_01_DE.md`, section 7.

22. Parce que les réunir est une transformation : décider que les deux champs mesurent la même chose, choisir un nom, écrire la règle. Cette décision appartient à l'ingénieur analytique, dans du code relu. Le bronze garde ce que la source a livré : 88 678 messages avec `temp_moteur_max`, 69 468 avec `temperature_moteur_max`.

Théorie : `Theorie_02_DE.md`, annexe A.3.

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

P1.

```sql
SELECT row_group_num_rows, stats_min, stats_max
FROM parquet_metadata('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet')
WHERE path_in_schema = 'Timestamp';
```

Résultat : 4 106 lignes, de 2025-10-27 00:00:00 à 2025-10-28 23:58:00. La requête ne lit que le pied de page : aucune donnée n'est décompressée. Le minimum tombe la veille parce que le fichier du 28 octobre commence par les valeurs de départ, la dernière valeur retenue de chaque étiquette le 27 ; pour une étiquette qui n'a pas bougé du jour, c'est le point archivé de force à minuit.

Théorie : `Theorie_02_DE.md`, sections 3 et 5.

P2.

```sql
SELECT path_in_schema, total_compressed_size, total_uncompressed_size
FROM parquet_metadata('plateforme/lac/bronze/usine/historien/jour=2025-10-28/*.parquet');
```

Résultat, en octets compressés puis non compressés : `TagName` 3 260 et 4 568, `Timestamp` 4 529 et 9 122, `Value` 12 598 et 32 742, `Quality` 94 et 132. `Value` pèse le plus : des nombres à virgule qui varient sans cesse, où ni le dictionnaire ni les plages ne trouvent de répétition. `Quality` pèse le moins : presque toujours 192, une longue plage que le codage écrit en quelques octets.

Théorie : `Theorie_02_DE.md`, section 3.

P3.

```sql
EXPLAIN ANALYZE
SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'
WHERE jour = DATE '2025-10-01';

EXPLAIN ANALYZE
SELECT COUNT(*) FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'
WHERE Timestamp >= TIMESTAMP '2025-10-01' AND Timestamp < TIMESTAMP '2025-10-02';
```

Le filtre sur `jour` lit un fichier (`Scanning Files: 1/92`) ; le filtre sur `Timestamp` en lit 92 (`Total Files Read: 92`), parce que l'horodatage n'est pas dans le chemin. `EXPLAIN ANALYZE` exécute la requête et rend son plan, avec ce que chaque opérateur a lu ; la lecture d'un plan est au Cours 10 de la formation SQL avancé, section 3, et ici on n'en lit que le nombre de fichiers. Les deux rendent 3 714 lignes, mais pas les mêmes :

```sql
SELECT jour, COUNT(*) AS lignes
FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'
WHERE Timestamp >= TIMESTAMP '2025-10-01' AND Timestamp < TIMESTAMP '2025-10-02'
GROUP BY jour
ORDER BY jour;
```


© 2026 iAlexMG. Tous droits réservés.

Résultat : 3 656 lignes dans `jour=2025-10-01` et 58 dans `jour=2025-10-02`. Le filtre sur l'horodatage prend les points mesurés le 1er octobre, dont les 58 valeurs de départ du fichier du 2, et laisse de côté les 58 valeurs de départ datées du 30 septembre que contient le fichier du 1er. Le filtre sur la partition prend le fichier livré, valeurs de départ comprises. Une par étiquette de chaque côté : les comptes tombent égaux par hasard. Deux questions différentes, « ce qui a été mesuré ce jour-là » et « ce qui a été livré ce jour-là », et deux filtres différents.

Théorie : `Theorie_02_DE.md`, section 5.

P4.

```
mkdir essais
```

```
COPY (
    SELECT TagName, Timestamp, Value, Quality
    FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet'
    ORDER BY TagName, Timestamp
) TO 'essais/historien_trie.parquet' (FORMAT parquet, COMPRESSION zstd);

SELECT row_group_id, row_group_num_rows, stats_min, stats_max
FROM parquet_metadata('essais/historien_trie.parquet')
WHERE path_in_schema = 'TagName'
ORDER BY row_group_id;
```

Trois groupes : le premier de 122 880 lignes, de `ESS.BANC.REGIME` à `LMN.VIS.COUPLE` ; le deuxième de 122 880, de `LMN.VIS.COUPLE` à `UTL.CMP2.TEMP_REFOULEMENT` ; le troisième de 29 676, de `UTL.CMP2.TEMP_REFOULEMENT` à `UTL.SEC.POINT_ROSEE`. `UTL.CMP2.VIBRATION` se range après `UTL.CMP2.TEMP_REFOULEMENT` : seul le troisième groupe peut la contenir, et les deux autres se sautent.

Le fichier trié pèse 1 303 512 octets, plus que le fichier non trié (1,2 Mo). Trier par étiquette casse l'ordre du temps : dans le fichier d'origine, les horodatages voisins se suivent et se compressent très bien ; triés par étiquette, ils repartent au début de la période à chaque nouvelle étiquette. Trier sert un filtre, et peut coûter en taille : on trie pour les requêtes qu'on fait vraiment.

Théorie : `Theorie_02_DE.md`, section 3.

P5.

```
COPY (SELECT * FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet')
TO 'essais/telemetrie_compacte.parquet' (FORMAT parquet, COMPRESSION zstd);

SELECT size FROM read_blob('essais/telemetrie_compacte.parquet');

.timer on

SELECT COUNT(DISTINCT message.sernr) FROM 'plateforme/lac/bronze/telemetrie/*/*.parquet';
SELECT COUNT(DISTINCT message.sernr) FROM 'essais/telemetrie_compacte.parquet';
```


© 2026 iAlexMG. Tous droits réservés.

Le fichier compacté pèse 5 678 912 octets, contre 10 817 501 pour les 843 fichiers : chaque petit fichier répétait son pied de page, son schéma et ses dictionnaires. Les deux requêtes rendent 6 759 véhicules, en environ 0,15 seconde sur les 843 fichiers et 0,012 seconde sur le fichier compacté : plus de dix fois moins. Même contenu, même format, seul le nombre de fichiers a changé. Le compactage d'un vrai lac se fait sans casser les lectures en cours, ce que le Cours 10 confie à un format de table.

Théorie : `Theorie_02_DE.md`, section 5.

Diagnostics

E1. Le bronze dérive des sources, et il ne suit pas tant qu'on ne relance pas le script qui le produit. Le générateur a réécrit 731 fichiers d'historien ; le bronze contient toujours les 92 de la conversion précédente. Personne n'a vu d'erreur parce qu'aucune n'est levée : l'étude lit un bronze valide, simplement périmé. Le correctif : relancer `bronze_historien.sql` après toute regénération, et regarder son contrôle, qui rendrait 2 133 145 lignes côté CSV contre 275 436 côté Parquet si l'on oubliait. C'est la raison d'être du contrôle en fin de script : un piège qui ne plante pas reçoit une requête qui montre l'écart.

Théorie : `Theorie_02_DE.md`, sections 6 et 7.

E2. La clé a une cardinalité beaucoup trop forte. 58 étiquettes sur 92 jours font 5 336 partitions, qui contiennent une cinquantaine de lignes en moyenne, de quelques lignes pour un état de marche à quelques centaines pour une mesure bavarde : le coût fixe de chaque fichier écrase le gain de l'élagage. Une requête sur un mois ouvre près de 1 800 fichiers au lieu de 30. Filtrer par étiquette ne demande pas une partition : trier les lignes par étiquette dans chaque fichier quotidien rend les statistiques utiles (exercice P4), sans multiplier les fichiers. Le correctif : revenir à une partition par jour livré, et trier à l'intérieur si le filtre par étiquette est vraiment fréquent.

Théorie : `Theorie_02_DE.md`, sections 3 et 5.

Questions à développer

A. Le premier écart, de 3 secondes à 0,6, revient presque entièrement à la détection des types : quand on laisse DuckDB deviner, elle se refait à chaque requête, avant la lecture proprement dite. Déclarer les types supprime ce travail, et c'est la même décision qui corrige la justesse des colonnes. Le second écart, de 0,6 à 0,016 seconde, revient au format. Le CSV oblige à lire chaque octet des 13,7 Mo, à découper chaque ligne et à convertir chaque valeur de texte en nombre ou en horodatage. Le Parquet ne lit que les blocs de `TagName` et de `Value`, déjà typés et compressés (1,5 Mo pour tout le bronze), et comparer des étiquettes encodées par dictionnaire coûte peu. La leçon : le format rapporte beaucoup, mais une bonne partie de la lenteur d'un lac en CSV tient à ce qu'on laisse deviner, et se corrige sans changer de format.

Théorie : `Theorie_02_DE.md`, sections 1, 3 et 6.

B. Tout typer dès le bronze, c'est convertir sans pouvoir revenir en arrière, avant que quiconque ait écrit la règle. Premier exemple : ramener `debut` et `fin` à UTC jette le décalage, et avec lui le fuseau où roulait le véhicule ; l'ingénieur analytique qui voudra un jour l'heure locale d'une sortie ne pourra plus la reconstruire, puisque le bronze était la seule copie fidèle. Deuxième exemple : convertir les dates SAP suppose une règle pour `00000000`, la date vide d'`AEDAT` ; nulle, ou une date fictive, c'est une décision métier, pas une conversion technique. Troisième exemple : un code comme `VTWEG` converti en entier perdrait les zéros d'un code `01`. On ne facilite pas la vie de l'ingénieur analytique en décidant à sa place ; on la facilite en lui livrant la source intacte et documentée. Ce qui se convertit sans rien perdre se type au bronze, comme l'horodatage sans fuseau de l'historien. Le reste attend la couche où la règle s'écrit, se teste et se relit.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_02_DE.md`, section 4.

C. Le grain de livraison est l'extrait mensuel : c'est ce qu'on remplacera d'un bloc si l'équipe de l'ERP en relivre un. Une partition par extrait, `extrait=2024-05/`, avec la table dans le chemin (`erp/vbak/extrait=2024-05/`), respecte ce grain. Le filtre fréquent n'est pas l'extrait, mais la date de création ou de livraison des commandes ; il se sert par les statistiques ou dans la couche suivante, pas par le chemin. La cardinalité reste faible : 48 extraits en quatre ans. L'extrait `2023-03_bis` reçoit sa propre partition, `extrait=2023-03_bis/`, parce qu'il a été livré : le bronze garde ce qui arrive, et c'est un contrôle (Cours 11) qui signale son contenu identique à `2023-03`, pas un tri silencieux à l'ingestion. La décision du Cours 03, pour l'ERP, sera de charger ces extraits dans la couche `RAW` en texte, marqués par leur lot : la même logique, dans l'entrepôt.

Théorie : `Theorie_02_DE.md`, section 5.

Mises en situation

M1. Transport collectif des Deux-Rives, le bronze.

Choix	Décision	Pourquoi
Format	Parquet, avec des types déclarés	des lectures par date et par ligne sur des millions de messages ; le JSON reste la forme reçue, archivée à part s'il faut la garder
Compression	zstd	un bronze lu bien plus souvent qu'écrit, et une relecture annuelle où chaque octet compte
Clé de partition	le jour de réception	le grain de livraison, le filtre fréquent, et 365 partitions par an ; la ligne d'autobus n'entre pas dans le chemin
Rangement dans le fichier	trié par ligne d'autobus, puis par heure	les statistiques servent le filtre par ligne sans multiplier les fichiers
Taille des fichiers	quelques fichiers par jour au plus	deux millions de messages par jour tiennent en un ou deux fichiers de taille raisonnable

Les fichiers écrits toutes les cinq minutes font 288 petits fichiers par jour : c'est le piège des petits fichiers, fabriqué exprès. Deux réponses défendables : laisser le service écrire ses petits fichiers dans une zone d'arrivée, puis compacter chaque nuit la journée précédente en un fichier trié dans le bronze ; ou écrire directement l'heure en cours dans un fichier ouvert et le fermer à l'heure pile, pour 24 fichiers par jour. Serait acceptable aussi : partitionner par mois si les fichiers quotidiens restent petits. Serait indéfendable : partitionner par autobus et par jour, soit 87 600 partitions par an.

Théorie : `Theorie_02_DE.md`, sections 3 et 5.

M2. Coopérative électrique de la Rivière-aux-Saules, le bronze.

© 2026 iAlexMG. Tous droits réservés.

Colonne	Type dans le bronze	Pourquoi
numéro de compteur	texte	un identifiant, qui peut porter des zéros de tête
fin d'intervalle, avec décalage	texte	le décalage est la seule trace de l'heure que le compteur affichait sur place ; le bronze garde ce qu'il a reçu
énergie en kWh, trois décimales	`DECIMAL(12, 3)`	une quantité facturée : exacte, pas approchée
qualité	texte	un code (`R`, `E`), pas un nombre

La partition : le jour de livraison, puisque le système livre un fichier par nuit et qu'on voudra remplacer une nuit reprise. À 1,7 million de lectures par jour, un fichier quotidien reste de taille raisonnable. Ce qui tournerait mal en laissant deviner : les numéros de compteur deviendraient des entiers si aucun ne commence par zéro dans l'échantillon, puis une livraison suivante casserait ; et l'énergie passerait en `DOUBLE`, avec des sommes facturées au millième près plutôt que justes.

La fin d'intervalle, elle, ne poserait pas le problème qu'on croit. Ramenée à UTC, elle reste juste : les deux intervalles de 1 h du retour à l'heure normale portent deux décalages différents, `-04:00` et `-05:00`, donc deux instants distincts à une heure d'écart. Ce que la conversion perd est l'heure affichée sur place, et c'est pour cela qu'on garde le texte reçu au bronze. Le vrai piège de l'heure est ailleurs : une heure locale livrée sans aucun décalage, comme celle de l'historien de l'usine, où les deux 1 h sont vraiment indémêlables. Le Cours 08 s'y attaque.

Théorie : `Theorie_02_DE.md`, sections 4 et 5.

M3. (avancé) Le lac de Boréal dans le nuage. Le coût se mesure en requêtes et en octets lus, donc le nombre de fichiers compte autant que leur taille. Proposition : garder l'historien par jour, qui ne fait que 92 fichiers, et ne rien changer à sa logique de remplacement. Pour la télémétrie, séparer deux zones. Les trente derniers jours restent par jour de réception, parce que c'est ce que les analystes lisent et ce qu'on remplace quand un jour est relivré. Au-delà de trente jours, un compactage mensuel réécrit les jours d'un mois en un seul fichier trié par numéro de série, dans une partition par mois : environ 30 fichiers par an au lieu de 365, et une relecture hebdomadaire de tout l'historique qui ouvre quelques dizaines de fichiers au lieu de plus de 800. Les messages en retard, qui arrivent après le compactage d'un mois, attendent le compactage suivant ou déclenchent la réécriture du mois.

Ce qui ferait changer d'avis : des relivraisons fréquentes de vieux jours, qui rendraient la réécriture des mois compactés coûteuse ; ou une plateforme qui compacte d'elle-même sans casser les lectures, comme un format de table ouvert (Cours 10) ou l'optimisation automatique de Databricks, qui rendrait le plan à la main inutile.

Théorie : `Theorie_02_DE.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.